Scripst Documentation

Article Style Set

AnZrew AnZreww AnZrewww

2026-10-20

Abstract: Scripst is a simple and easy-to-use Typst language template, suitable for various scenarios such as daily documents, assignments, notes, papers, etc.

Keywords: Scripst, Typst, template

Contents

1 Typesetting Typst Documents with Scripst 3
1.1 Using Typst 3
1.2 Using Scripst 3
1.2.1 Online Usage 4
1.2.2 Offline Usage 4
2 Template Parameter Description 6
2.1 template 7
2.2 title 7
2.3 info 7
2.4 author 8
2.5 time 8
2.6 abstract 8
2.7 keywords 9
2.8 fonts: configure fonts by role 10
2.9 font-size 11
2.10 contents 12
2.11 content-depth 12
2.12 matheq-depth 12
2.13 counter-depth 12
2.14 cb-counter-depth 13
2.15 countblocks 13

2.16 countblock-lang . . . 13
2.17 matheq-outline . . . 13
2.18 counter-outline . . . 13
2.19 matheq-color / counter-color . . . 14
2.20 link-color . . . 14
2.21 ref-color . . . 14
2.22 header . . . 14
2.23 lang . . . 14
2.24 par-indent . . . 15
2.25 par-leading . . . 15
2.26 par-spacing . . . 15
2.27 numbering-format . . . 16
2.28 chapter-numbering-format . . . 16
2.29 offset . . . 16
2.30 body . . . 16

3 Template Effect Display . . . 17

3.1 Front Page . . . 17
3.2 Table of Contents . . . 17
3.3 Fonts and Environments . . . 17
3.3.1 Fonts . . . 17
3.3.2 Environments . . . 17
3.3.3 Lists . . . 19
3.3.4 Quotes . . . 19
3.3.5 Links . . . 19
3.3.6 Hyperlinks and Citations . . . 20
3.4 `#newpara()` Function . . . 20
3.5 labelset . . . 20
3.6 Unified numbering powered by Ratchet . . . 22
3.7 countblock . . . 22
3.7.1 Default Countblocks . . . 22
3.7.2 Changing the depth of all countblocks . . . 26
3.7.3 Changing an individual countblock depth . . . 26
3.7.4 Adding a new countblock . . . 27
3.7.5 The `cb` registry structure . . . 28
3.7.6 Using countblock . . . 28

3.7.7 Summary . . . 30
3.8 Some Other Blocks . . . 31
3.8.1 Blank Block . . . 31
3.8.2 Proof and ■ (Quod Erat Demonstrandum) . . . 31
3.8.3 Solution . . . 31
3.8.4 Separator . . . 31

4 Conclusion . . . 32

Typst is a simple document generation language with syntax similar to lightweight Markdown markup. Using appropriate `set` and `show` commands, you can highly customise the style of your documents.

Scripst is a simple and easy-to-use Typst language template, suitable for various scenarios such as daily documents, assignments, notes, papers, etc.

1 Typesetting Typst Documents with Scripst

1.1 Using Typst

Typst is a lighter language to use compared to LaTeX. Once the template is written, you can complete the document writing with lightweight markup similar to Markdown.

Compared to LaTeX, Typst has the following advantages:

- Extremely fast compilation speed
- Simple and lightweight syntax
- Strong code extensibility
- Easier mathematical formula input
- ...

Therefore, Typst is very suitable for writing lightweight daily documents. You can get even better typesetting results than LaTeX with the time cost of writing Markdown.

You can install Typst in the following ways:

```
sudo apt install typst # Debian/Ubuntu
sudo pacman -S typst # Arch Linux
winget install --id Typst.Typst # Windows
brew install typst # macOS
```

You can also find more information in the Typst GitHub repository.

1.2 Using Scripst

Based on Typst, Scripst provides some simple templates for convenient daily document generation.

1.2.1 Online Usage

Scripst Package has already been submitted to the community. If network available, you can directly use

```
#import "@preview/scripst:1.2.0": *
```

to import the Scripst templates in your document.

You can also use `typst init` to create a new project with the template:

```
typst init @preview/scripst:1.2.0 project_name
```

This method does not require downloading the template files, just import them in the document.

1.2.2 Offline Usage

Using extracted files

You can find and download the Scripst templates in the Scripst GitHub repository.

You can choose `<> code` → `Download ZIP` to download the Scripst templates. When using them, just place the template files in your document directory and import the template files at the beginning of your document.

⚠️

Consider the project directory structure to correctly import the template files.

```
project/
├─── src/
│    ├─── main.typ
│    ├─── ...
│    └─── components.typ
├─── pic/
│    ├─── ...
├─── main.typ
├─── chap1.typ
├─── chap2.typ
├─── ...
```

If the project directory structure is as shown above, then the way to import the template files in `main.typ` should be:

```
#import "src/main.typ": *
```

The advantage of this method is that you can adjust some parameters in the template at any time. Since the template is designed modularly, you can easily find and modify the parts you need to change.

Local package management

A better way is to refer to the official local package management documentation and place the template files in the local package management directory `{data-dir}/typst/packages/{namespace}/{name}/{version}`, so you can use the Scripst templates anywhere.

Of course, you don't have to worry about not being able to modify the template files. You can directly use `#set, #show` commands in the document to override some parameters in the template.

For example, the template should be placed in

```
~/.local/share/typst/packages/preview/scripst/1.2.0                # in
Linux
%APPDATA%\typst\packages\preview\scripst\1.2.0                     # in
Windows
~/Library/Application Support/typst/packages/preview/scripst/1.2.0  #
macOS
```

You can execute the following command:

```
cd ~/.local/share/typst/packages/preview/scripst/
git clone --branch v1.2.0 --depth 1 https://github.com/An-314/scripst.
git 1.2.0
```

If the directory structure is like this, then the way to import the template files in the document should be:

```
#import "@preview/scripst:1.2.0": *
```

The advantage of this is that you can directly use `typst init` to create a new project with the template:

```
typst init @preview/scripst:1.2.0 project_name
```

After importing the template, create an `article` file in this way:

```
#show: scripst.with(
  title: [How to Use Scripst],
  info: [This is the article template],
```

```
  author: ("Author1", "Author2", "Author3"),
  time: datetime.today().display(),
  abstract: [Abstract],
  keywords: ("Keyword1", "Keyword2", "Keyword3"),
  contents: true,
  content-depth: 3,
  matheq-depth: 2,
  lang: "en",
)
```

See Section 2 for the meaning of these parameters.

Then you can start writing your document.

2 Template Parameter Description

Scripst template provides some parameters to customise the style of the document.

```
#let scripst(
  template: "article",          // "article", "book", "report",
"beamer"
  title: "",                    // str, content, none
  info: "",                     // str, content, none
  author: (),                   // str, content, array, none
  time: "",                     // str, content, none
  abstract: none,               // str, content, none
  keywords: (),                 // array
  preface: none,                // str, content, none
  font-size: auto,              // documents: 11pt; beamer: 20pt
  fonts: (:),                   // dictionary: font families by role
  beamer: (:),                  // dictionary: presentation options
  contents: false,              // bool
  content-depth: 2,             // int
  matheq-depth: 2,              // int: (1, 2, 3)
  counter-depth: 2,             // int: (1, 2, 3)
  cb-counter-depth: 2,          // int: (1, 2, 3)
  countblocks: cb,              // dictionary
  countblock-lang: "en",        // str
  matheq-outline: "(1.1)",       // str, function
  counter-outline: "1.1",       // str, function
  matheq-color: red,            // color
  counter-color: blue,          // color
  link-color: blue,             // color
  ref-color: red,               // color
  header: true,                 // bool
```

```
  lang: "zh",                    // str
  par-indent: auto,              // documents: 2em; beamer: 0em
  par-leading: none,             // length, none
  par-spacing: none,             // length, none
  numbering-format: none,        // str, function, none
  chapter-numbering-format: none, // str, function, none
  offset: 0,                     // int
  body,
) = {
  ...
}
```

2.1 template

Parameter	Type	Optional Values	Default Value	Description
template	`str`	`("article", "book", "report", "beamer")`	`"article"`	Template type

Scripst provides four templates: article, book, report, and beamer. Version 1.2.0 requires Typst 0.15.0 or newer.

This template uses the article template.

- article: Suitable for daily documents, assignments, tiny notes, light papers, etc.
- book: Suitable for books, course notes, etc.
- report: Suitable for lab reports, papers, etc.
- beamer: Academic presentations powered by Touying 0.8.0 / Stargazer. See `docs/beamer.md` for options and `docs/locale/beamer-en.typ` for a complete example.

Passing other strings will cause a `panic`: `"Unknown template!"`.

2.2 title

Parameter	Type	Default Value	Description
title	`content`, `str`, `none`	`""`	Document title

The title of the document. (If not empty) it will appear at the beginning and in the header of the document.

2.3 info

Parameter	Type	Default Value	Description
info	`content`, `str`, `none`	`""`	Document information

The information of the document. (If not empty) it will appear at the beginning and in the header of the document. It can be used as a subtitle or supplementary information for the article.

2.4 author

Parameter	Type	Default Value	Description
author	`str`, `content`, `array`, `none`	`()`	Document authors

The authors of the document. Pass a list of `str` or `content`. Or, simply pass a `str` or `content` object.

> **Note**
>
> Note, if there is only one author, you can simply pass a `str` or `content`, and for multiple authors, a list of one `str` or `content`, for example: `author: ("Author I", "Author II")`

It will be displayed at the beginning of the article with $\min(\#\text{authors}, 3)$ authors in a line.

2.5 time

Parameter	Type	Default Value	Description
time	`content`, `str`, `none`	`""`	Document time

The time of the document. It will appear at the beginning and in the header of the document.

You can choose to use Typst's `datetime` to get or format the time, such as today's date:

```
datetime.today().display()
```

2.6 abstract

Parameter	Type	Default Value	Description
abstract	`content`, `str`, `none`	`none`	Document abstract

The abstract of the document. (If not empty) it will appear at the beginning of the document.

It is recommended to define a `content` before using the abstract, for example:

```
#let abstract = [
  This is a simple document template used to generate simple daily
documents to meet the needs of documents, assignments, notes, papers,
```

```
etc.
]

#show: scripst.with(
  ...
  abstract: abstract,
  ...
)
```

Then pass it to the `abstract` parameter.

2.7 keywords

Parameter	Type	Default Value	Description
keywords	`array`	`()`	Document keywords

The keywords of the document. Pass a list of `str` or `content`.

Like `author`, the parameter is a list, not a string.

Keywords will only appear at the beginning of the document if `abstract` is not empty.

2.8 fonts: configure fonts by role

Starting with Scripst 1.2.0, the main function accepts a `fonts` dictionary. Its default, `(:)`, keeps the existing fonts. Supply only the roles you want to change; there is no need to edit package files. `font-size` still controls the text size. Inline and block code inherit the surrounding text size instead of using a fixed `10pt` or an extra reduction. Local `#set text(size: ...)` rules also apply. To shrink code independently, add `#show raw: set text(size: 9pt)` after the template show rule, or use `raw.where(block: true)` to target block code only.

```
#import "@preview/scripst:1.2.0": *
#show: scripst.with(
  fonts: (
    body: ("Libertinus Serif", "Noto Serif CJK SC"),
    heading: ("Libertinus Sans", "Noto Sans CJK SC"),
    countblock: ("Libertinus Serif", "Noto Serif CJK SC"),
    raw: ("DejaVu Sans Mono", "Noto Sans CJK SC"),
    math: "New Computer Modern Math",
  ),
)
```

Each value accepts a font name, a font descriptor such as `(name: "SimSun", covers: "latin-in-cjk")`, or a non-empty array of these. Typst tries fonts in array order for each character. Install fonts locally or upload them to your web project; Scripst does not bundle fonts.

The default lists below are abbreviated as:

- S: `("CMU Serif", "Linux Libertine", "SimSun")`
- H: `("CMU Serif", "Linux Libertine", "SIMHEI")`
- K: `("CMU Serif", "Linux Libertine", "KaiTi")`

Key	Role	Default / inheritance
`body`	Body text, ordinary lists, page numbers	S
`title`	Document / cover title	H
`info`	Subtitle	Follows `author` in article; `title` in book/report/beamer
`author`	Authors	K
`time`	Date	Follows `body`
`abstract`	Abstract body	Follows `body`
`keywords`	Keyword contents	Follows `emph`
`preface`	Preface title and body	Follows `body`
`contents`	Base font for outline entries	Follows `body`
`heading`	Headings, including outline title	H

Key	Role	Default / inheritance
`countblock`	Theorem-like block bodies and `blankblock`	S
`caption`	Captions and figure base font	K
`table`	Table contents	Follows `body`
`header`	Document headers; beamer footer title	`("Linux Libertine", "SimSun")`
`strong`	Bold text, block titles, top-level outline entries	H
`emph`	Emphasis, including proof/solution titles	K
`quote`	Quotations, retaining italics	K
`raw`	Inline and block code	`("Consolas", "SimSun")`
`math`	Inline and display equations	`auto`: keep Typst's math font settings

Changing `body` does not replace independent defaults such as `heading`, `strong`, `emph`, or `countblock`. Roles marked "follows" inherit the resolved parent font unless explicitly overridden. `auto` restores a role's default or inheritance. More specific formatting still applies inside components: block titles use `strong`, equations use `math`, and the outline title uses `heading`. Set related roles to the same font list when a uniform appearance is desired.

The exported `default-fonts` dictionary contains all defaults. The existing `font` dictionary remains available and may also be passed to `fonts`.

```
#let family = ("Libertinus Serif", "Noto Serif CJK SC")
#let my-fonts = default-fonts + (
  body: family, heading: family, strong: family, emph: family,
  countblock: family,
)
#show: scripst.with(fonts: my-fonts)
```

Use an OpenType MATH font for `math`. Explicit `raw` lists are passed through unchanged; the default SimSun fallback is not appended to your override. Unknown keys, empty lists, and invalid value types produce an error.

2.9 font-size

Parameter	Type	Default Value	Description
font-size	`length` / `auto`	`auto`	Document font size

The default `auto` uses `11pt` for article/book/report and `20pt` for beamer.

Refer to the `length` type values, you can pass `pt`, `mm`, `cm`, `in`, `em`, etc.

2.10 contents

Parameter	Type	Default Value	Description
contents	`bool`	`false`	Whether to generate a table of contents

Whether to generate a table of contents. The default is `false`.

2.11 content-depth

Parameter	Type	Default Value	Description
content-depth	`int`	`2`	Depth of the table of contents

The depth of the table of contents. The default is `2`.

2.12 matheq-depth

Parameter	Type	Optional Values	Default Value	Description
matheq-depth	`int`	`1`, `2`, `3`	`2`	Depth of math equation numbering

The depth of math equation numbering. The default is `2`.

> **Note**
>
> For detailed behavior of counters, see Section 2.13.

2.13 counter-depth

Parameter	Type	Optional Values	Default	Description
counter-depth	`int`	`1`, `2`, `3`	`2`	Counter depth

The counter depth for images (`image`), tables (`table`), and code blocks (`raw`) within `figure` environments. Default is `2`.

> **Note Counter Details**
>
> When a counter has depth `1`, its numbering will be global and unaffected by chapters/sections, i.e., `1`, `2`, `3`, ...
>
> When a counter has depth `2`, its numbering follows level-1 headings, i.e., `1.1`, `1.2`, `2.1`, `2.2`, ... Before any level-1 heading, the missing chapter level is padded with `0`, giving `0.1`, `0.2`, etc.

When a counter has depth `3`, its numbering follows level-1 and level-2 headings, i.e., `1.1.1`, `1.1.2`, `1.2.1`, `1.2.2`, `2.1.1`, ... In chapter 1 before a level-2 heading, numbering starts at `1.0.1`; before any chapter headings, it starts at `0.0.1`.

Ratchet does not automatically reduce the depth when headings are missing. For short documents or slides needing global `1`, `2`, ... numbering, explicitly set the relevant depth to `1`.

2.14 cb-counter-depth

Parameter	Type	Optional Values	Default	Description
cb-counter-depth	`int`	`1`, `2`, `3`	`2`	Counter depth for `countblock`

The default numbering depth for entries passed through `countblocks`. Per-counter depths configured with `set-countblock-depth` or `add-countblock(depth: ...)` take precedence. See Section 3.7.2 for details.

2.15 countblocks

The countblock registry used by the template. It defaults to `cb`. Pass an updated registry here before using custom countblocks so Ratchet can configure their numbering and references.

2.16 countblock-lang

The language used for built-in countblock names. It defaults to `"en"` and supports `en`, `zh`, `fr`, `es`, `ja`, `de`, `it`, `pt`, `ru`, `ko`, `ar`, `hi`, `tr`, `nl`, `sv`, `fi`, `da`, `no`, `pl`, `gr`, and `vi`.

A custom block's `info` may also be a language dictionary such as `(en: "Assumption", zh: "假设", de: "Annahme")`. If the selected key is absent, Scripst falls back to `en`, then to the first available value. Built-in translations are maintained together in `src/locale/countblock.typ`.

2.17 matheq-outline

The equation numbering pattern. It defaults to `"(1.1)"`, so displayed equations and their references include parentheses.

2.18 counter-outline

The numbering pattern for figures and countblocks. The default is `"1.1"`; it accepts a numbering string or function.

2.19 matheq-color / counter-color

`matheq-color` defaults to `red` and controls equation references. `counter-color` defaults to `blue` and controls figure and countblock references. Both accept a color.

2.20 link-color

The text color of hyperlinks. It defaults to `blue`. Equation, figure, and countblock reference colors remain controlled by `matheq-color` and `counter-color`.

PDF link borders and hover feedback are controlled by the PDF viewer. Typst currently cannot reliably customize their appearance.

2.21 ref-color

The text color of ordinary `@label` references. It defaults to `red`. Equation references remain controlled by `matheq-color`; figure and countblock references remain controlled by `counter-color`.

2.22 header

Parameter	Type	Default	Description
header	`bool`	`true`	Enable header

Whether to generate headers. Default is `true`.

Note

The header displays the document title, metadata, and current chapter/section title:

- If all three exist, they will be displayed in the header in three equal parts.
- If the document has no metadata, the header will show the title on the left and chapter title on the right.
  - If the document also lacks a title, only the chapter title will appear on the right.
- If the document has no level-1 headings, the header will show the title on the left and metadata on the right.
  - If there's no metadata, only the title will appear on the left.
- If none of these elements exist, the header will remain empty.

2.23 lang

Parameter	Type	Default Value	Description
lang	`str`	`"zh"`	Document language

The document language. The default is `"zh"`.

Accepts ISO_639-1 encoding format, such as `"zh"`, `"en"`, `"fr"`, etc.

2.24 par-indent

Parameter	Type	Default	Description
par-indent	`length` / `auto`	`auto`	First-line paragraph indentation

Controls first-line paragraph indentation. The default `auto` uses `2em` for article/book/report and `0em` for beamer. Set to `0em` to disable indentation explicitly.

2.25 par-leading

Parameter	Type	Default	Description
par-leading	`length`, `none`	`none`	Line spacing within paragraphs

Adjusts line spacing within paragraphs. In article/book/report, `none` selects the language-dependent value (`1em` for Chinese). In beamer, it selects `0.65em`. An explicit length overrides either default.

Note

Language-dependent defaults for article/book/report are shown below:

Script Category	Default
East Asian (Chinese/Japanese/Korean)	1em
South/Southeast Asian/Amharic (Thai/Hindi/etc.)	0.85em
Arabic Scripts (Arabic/Persian/etc.)	0.75em
Cyrillic Scripts (Russian/Bulgarian/etc.)	0.7em
Other Languages	0.6em

,

2.26 par-spacing

Parameter	Type	Default	Description
par-spacing	`length`, `none`	`none`	Vertical spacing between paragraphs

Sets vertical spacing between paragraphs. In article/book/report, `none` selects the language-dependent value (`1.2em` for Chinese). In beamer, it selects `0.7em`. An explicit length overrides either default.

Note

Language-dependent defaults for article/book/report are shown below:

Script Category	Default
East Asian (Chinese/Japanese/Korean)	1.2em
South/Southeast Asian/Amharic (Thai/Hindi/etc.)	1.3em
Arabic Scripts (Arabic/Persian/etc.)	1.25em
Cyrillic Scripts (Russian/Bulgarian/etc.)	1.2em
Other Languages	1em

,

2.27 numbering-format

The heading-numbering string or function. The main function defaults to `none`: article/book/report resolve it to `"1.1"`, while beamer leaves visible headings unnumbered. For document templates, string patterns apply `offset`; a custom function is responsible for its own displayed numbers.

2.28 chapter-numbering-format

The level-1 heading format for article/book/report. The default `none` chooses a language-dependent format. Beamer does not use this parameter. Use a function when a title contains literal text that would otherwise be interpreted as numbering placeholders:

```
#show: scripst.with(
  chapter-numbering-format: (n, ..rest) => "Unit " + str(n),
)
```

2.29 offset

An integer chapter offset, default `0`, with a minimum of `-1`. Document templates use it for heading-numbering strings. Beamer passes it to Ratchet's chapter prefixes; it does not rewrite a custom visible-heading format.

2.30 body

When using `#show: scripst.with(...)`, the `body` parameter does not need to be passed manually. Typst will automatically pass the remaining document content to the `body` parameter.

3 Template Effect Display

3.1 Front Page

The beginning of the document will display the title, information, authors, time, abstract, keywords, etc., as shown at the beginning of this document.

3.2 Table of Contents

If the `contents` parameter is `true`, a table of contents will be generated, as shown in this document.

3.3 Fonts and Environments

Scripst provides some commonly used fonts and environments, such as bold, italic, headings, images, tables, lists, quotes, links, math formulas, etc.

3.3.1 Fonts

This is normal text. C'est un texte normal.

This is bold text. C'est un texte en gras.

This is italic text. C'est un texte en italique.

Install the CMU Serif font for better (LaTeX-like) display effects.

3.3.2 Environments

3.3.2.1 Headings

Level 1 headings are numbered according to the document language, including Chinese/ Roman numerals/Greek letters/Kana/Numerals in Arabic/Hindi numerals, etc. Other levels use Arabic numerals.

3.3.2.2 Images

The image environment will automatically number the images, as shown below:



Figure 3.3.1: Little Scara

3.3.2.3 Tables

Thanks to the `tablem` package, you can write tables in Markdown style when using this template, as shown below:

```
#figure(
  three-line-table[
    | Name | Age | Gender |
    | --- | --- | --- |
    | Jane | 18 | Male |
    | Doe | 19 | Female |
  ],
  caption: [`three-line-table` table example],
)
```

Name	Age	Gender
Jane	18	Male
Doe	19	Female

Table 3.3.1: `three-line-table` table example

```
#figure(
  tablem[
    | Name | Age | Gender |
    | --- | --- | --- |
    | Jane | 18 | Male |
    | Doe | 19 | Female |
  ],
  caption: [`tablem` table example],
)
```

Name	Age	Gender
Jane	18	Male
Doe	19	Female

Table 3.3.2: `tablem` table example

You can choose `numbering: none,` to make the table unnumbered, as shown above, the tables in the previous chapters did not enter the full text table counter.

3.3.2.4 Math Formulas

Math formulas have inline and block modes.

Inline formula: $a^2 + b^2 = c^2$.

Block formula:

$$a^2 + b^2 = c^2$$
$$\frac{1}{2} + \frac{1}{3} = \frac{5}{6} \tag{3.1}$$

are numbered.

Thanks to the `physica` package, Typst's math input method is greatly expanded while still retaining its simplicity:

$$\begin{aligned} \nabla \cdot \boldsymbol{E} &= \frac{\rho}{\varepsilon_0} \\ \nabla \cdot \boldsymbol{B} &= 0 \\ \nabla \times \boldsymbol{E} &= -\frac{\partial \boldsymbol{B}}{\partial t} \\ \nabla \times \boldsymbol{B} &= \mu_0 \left(\boldsymbol{J} + \varepsilon_0 \frac{\partial \boldsymbol{E}}{\partial t} \right) \end{aligned} \tag{3.2}$$

3.3.3 Lists

Typst provides a simple environment for lists, as shown:

```
- First item
- Second item
- Third item
```

- First item
- Second item
- Third item

```
+  First item
3. Second item
+  Third item
```

1. First item
3. Second item
4. Third item

```
/ First item: 1
/ Second item: 2
/ Third item: 23
```

First item 1
Second item 2
Third item 3

3.3.4 Quotes

```
#quote(attribution: "Einstein",
block: true)[
  God does not play dice with the
universe.
]
```

> *God does not play dice with the universe.*
>
> — *Einstein*

3.3.5 Links

```
#link("https://www.google.com/")
[Google]
```

Google

3.3.6 Hyperlinks and Citations

Use `<label>` and `@label` to achieve hyperlinks and citations.

3.4 `#newpara()` Function

By default, some modules do not automatically wrap. This is necessary, for example, if the explanation of the above math formula does not wrap.

But sometimes we need to wrap, and this is where the `#newpara()` function comes in.

Unlike the official `#parbreak()` function, the `#newpara()` function inserts a blank line between paragraphs, so it will start a new natural paragraph in any scenario.

Whenever you feel the need to wrap, you can use the `#newpara()` function.

3.5 labelset

Thanks to the `label` function in Typst, in addition to adding labels to this type, you can conveniently set styles for referenced objects using `label`.

Therefore, Scripst provides some commonly used settings, and you can set styles by simply adding a label.

```
== Schrödinger equation <hd.x>
The Schrödinger equation is as follows:
$
  i hbar dv(,t) ket(Psi(t)) = hat(H) ket(Psi(t))
$ <text.blue>
where
$
  ket(Psi(t)) = sum_n c_n ket(phi_n)
$ <eq.c>
is the wave function. From this, we can derive the time-independent
Schrödinger equation:
$
  hat(H) ket(Psi(t)) = E ket(Psi(t))
$
<text.teal>
where $E$<text.red> is #[energy]<text.lime>.
```

Schrödinger equation

The Schrödinger equation is as follows:

$$i\hbar\frac{\mathrm{d}}{\mathrm{d}t}|\Psi(t)\rangle = \hat{H}|\Psi(t)\rangle \tag{3.3}$$

where

$$|\Psi(t)\rangle = \sum_n c_n|\varphi_n\rangle$$

is the wave function. From this, we can derive the time-independent Schrödinger equation:

$$\hat{H}|\Psi(t)\rangle = E|\Psi(t)\rangle \tag{3.4}$$

where E is energy.

Currently, Scripst provides the following settings:

Label	Function
`eq.c`	Removes numbering from equations in math environments
`hd.c`	Removes numbering from headings but still displays them in the table of contents
`hd.x`	Removes numbering from headings and hides them in the table of contents
`text.{color}`	Sets the text color `color in (black, gray, silver, white, navy, blue, aqua, teal, eastern, purple, fuchsia, maroon, red, orange, yellow, olive, green, lime,)`

Table 3.6.1: Label Set



Note that the strings above have been used for styling settings. You can override their styles, but do not use these strings when working with labels and references.

3.6 Unified numbering powered by Ratchet

Scripst 1.2.0 uses Ratchet 0.0.4 as its unified numbering engine. Ratchet manages numbering, heading-level resets, and cross-references for equations, figures, tables, raw blocks, and custom `figure(kind: ...)` families. Every Scripst countblock is built on the same mechanism.

This integration keeps displayed numbers, references, and outline entries on one configuration. Each counter family can independently use depth `1`, `2`, or `3`, and new countblocks no longer require handwritten registration or heading-reset rules.

Note

Scripst configures Ratchet automatically, so no additional import is needed. Ratchet can also be used as a standalone package in other projects; see its Universe page.

3.7 countblock

Definition 3.1 countblock

Countblock is a counter module provided by Scripst for numbering countable elements in documents.

What you're seeing now is a `definition` block, which serves as an example of a counter module.

3.7.1 Default Countblocks

Scripst provides the following countblocks. Depth `2` means numbering follows level-1 headings; every entry initially inherits `cb-counter-depth: 2`.

Block	cb name	counter-name	Default depth	Color	Function
Definition	def	def	2	mycolor.green	#definition
Theorem	thm	thm	2	mycolor.blue	#theorem
Proposition	prop	prop	2	mycolor.violet	#proposition
Lemma	lem	prop	2	mycolor.violet-light	#lemma
Corollary	cor	prop	2	mycolor.violet-dark	#corollary
Remark	rmk	prop	2	mycolor.violet-darker	#remark
Claim	clm	prop	2	mycolor.violet-deep	#claim
Exercise	ex	ex	2	mycolor.purple	#exercise
Problem	prob	prob	2	mycolor.orange	#problem
Example	eg	eg	2	mycolor.cyan	#example
Note	note	note	2 (unnumbered by default)	mycolor.grey	#note
Caution	cau	cau	2 (unnumbered by default)	mycolor.red	#caution

Default Scripst countblock configuration

`proposition`, `lemma`, `corollary`, `remark`, and `claim` use the same `counter-name`, `"prop"`. They therefore share one number sequence and reset depth, while retaining separate titles and colors.

These functions share identical parameters and effects, differing only in counter names.

```
#definition(
  subname: "",
  count: true,
  lab: none,
  breakable: true,
)[
  ...
]
```

Parameter specifications:

Parameter	Type	Default	Description
subname	str, content	""	Entry name
count	bool	true	Enable numbering
lab	str	none	Entry label
breakable	bool	true	Allow this block instance to split across pages

Example usage:

```
#theorem(subname: [_Fermat's Last Theorem_], lab: "fermat")[

  No three $a, b, c in NN^+$ can satisfy the equation
  $
    a^n + b^n = c^n
  $
  for any integer value of $n$ greater than 2.
]
#proof[Cuius rei demonstrationem mirabilem sane detexi. Hanc marginis
exiguitas non caperet.]
```

This creates a numbered theorem block:

Theorem 3.1 ***Fermat's Last Theorem***

No three $a, b, c \in \mathbb{N}^+$ can satisfy the equation

$$a^n + b^n = c^n \tag{3.5}$$

for any integer value of n greater than 2.

Proof. Cuius rei demonstrationem mirabilem sane detexi. Hanc marginis exiguitas non caperet. ∎

3.7.1.1 `subname` Parameter

`subname` displays supplemental information after the counter, such as theorem names. In the example above, it shows "Fermat's Last Theorem".

3.7.1.2 `breakable` Parameter

Set `breakable` independently on each call, without changing `cb` or redefining a function. It defaults to `true`, allowing page breaks. With `false`, the entire block moves to the next page if the remaining space is insufficient. Keep `true` for content taller than a full page.

This option leaves the existing reference anchor structure unchanged: when an unbreakable block moves to the next page, its `lab` target may remain on the preceding page.

```
#theorem(breakable: false)[Keep this theorem on one page.]
#theorem[This theorem still allows page breaks by default.]
#note(breakable: false)[Keep this particular note together too.]
```

Custom blocks defined with `countblock.with(...)` accept the same per-call parameter.

3.7.1.3 `lab` Parameter

Use the `lab` parameter to create cross-references. For instance, reference the `fermat` theorem using `@fermat`:

```
Fermat never publicly proved @fermat.
```

Fermat never publicly proved Theorem 3.1.

Note that `proposition`, `lemma`, `corollary`, `remark`, and `claim` share the same counter:

Lemma 3.1

This is a lemma. Please prove it.

Proposition 3.2

This is a proposition. Please prove it.

Corollary 3.3

This is a corollary. Please prove it.

Remark 3.4

This is a remark. Please note it.

Claim 3.5

This is a claim. Please prove it.

Other counters operate independently.

3.7.1.4 `count` Parameter

Set `count: false` to disable numbering. `note` and `caution` default to `count: false`.

```
#note(count: true)[

  This is a numbered note.
]

#note[

  This is an unnumbered note.
]
```

Note 3.1

This is a numbered note.

Note

This is an unnumbered note.

3.7.2 Changing the depth of all countblocks

`cb-counter-depth` is the global default and accepts `1`, `2`, or `3`:

- `1`: continuous document-wide numbering;
- `2`: reset at level-1 headings;
- `3`: reset at level-1 and level-2 headings.

```
#show: scripst.with(
  countblocks: cb,
  cb-counter-depth: 3,
)
```

This changes every countblock that has no explicit depth. Per-counter overrides in `countblocks` take precedence.

3.7.3 Changing an individual countblock depth

Use `set-countblock-depth` to override one counter family:

```
#let blocks = set-countblock-depth(cb, "thm", 3)

#show: scripst.with(
  countblocks: blocks,
  cb-counter-depth: 2,
)
```

Only the theorem counter uses depth `3`; other independent counters inherit depth `2`.

```
#set-countblock-depth(cb, name, depth, detach: false)
```

Parameter	Type	Default	Description
`cb`	`dict`		Original registry
`name`	`str`		`cb` entry to configure
`depth`	`int`		New depth: `1`, `2`, or `3`
`detach`	`bool`	`false`	Detach this block from a shared counter

3.7.3.1 Shared counter families

Because `proposition`, `lemma`, `corollary`, `remark`, and `claim` all use `counter-name: "prop"`, the following changes the entire shared family:

```
#let blocks = set-countblock-depth(cb, "lem", 3)
```

Blocks sharing one counter must reset together and therefore cannot use different depths.

3.7.3.2 Detaching one block

To make only `lemma` use global depth `1` while the remaining `prop` family stays at depth `2`:

```
#let blocks = set-countblock-depth(cb, "lem", 1, detach: true)
#let lemma = countblock.with("lem", blocks)

#show: scripst.with(
  countblocks: blocks,
  cb-counter-depth: 2,
)
```

`detach: true` changes the lemma counter from `"prop"` to `"lem"`. Rewrap the function with the updated registry whenever its counter identity changes. A depth-only change to an already independent block such as `theorem` does not require rewrapping.

3.7.4 Adding a new countblock

Use `add-countblock`, then pass the updated registry through `countblocks`:

```
#let blocks = add-countblock(
  cb,
  "alg",
  "Algorithm",
  yellow,
  depth: 3,
)
#let algorithm = countblock.with("alg", blocks)

#show: scripst.with(
  countblocks: blocks,
  cb-counter-depth: 2,
)
```

`algorithm` now has an independent depth-3 counter while existing blocks retain the global depth.

```
#add-countblock(cb, name, info, color, counter-name: none, depth: none)
```

Parameter	Type	Default	Description
`cb`	`dict`		Original registry
`name`	`str`		New registry key
`info`	`str`, `content`, or `dict`		Displayed block title or localized titles
`color`	`color`		Background and left-border color
`counter-name`	`str`	`none`	Actual counter kind; defaults to `name`
`depth`	`int` or `none`	`none`	Explicit depth; `none` inherits `cb-counter-depth`

A new block can share an existing sequence by using the same `counter-name`:

```
#let blocks = add-countblock(
  cb,
  "asm",
  (en: "Assumption", zh: "假设"),
  aqua,
  counter-name: "thm",
)
#let assumption = countblock.with("asm", blocks)
```

All entries sharing a `counter-name` must resolve to the same depth. Scripst reports an error for conflicting configurations.

3.7.5 The `cb` registry structure

Each entry is `(info, color, counter-name, depth)`. The fourth field may be omitted or set to `none` to inherit `cb-counter-depth`:

```
#let cb = (
  "def": ("Definition", mycolor.green, "def", none),
  "thm": ("Theorem", mycolor.blue, "thm", none),
  "prop": ("Proposition", mycolor.violet, "prop", none),
  "lem": ("Lemma", mycolor.violet-light, "prop", none),
  // ...
  "cb-counter-depth": 2,
)
```

3.7.6 Using countblock

After defining a block, use the `countblock` function to create it:

```
#countblock(
  name,
  cb,
  subname: "",
```

```
  count: true,
  lab: none,
  breakable: true,
)[
  ...
]
```

Parameter specifications:

Parameter	Type	Default	Description
`name`	`str`		Counter name
`cb`	`dict`		Counter dictionary
`subname`	`str`, `content`	`""`	Entry name
`count`	`bool`	`true`	Enable numbering
`lab`	`str`	`none`	Reference label
`breakable`	`bool`	`true`	Allow this block instance to split across pages

- `name`: Counter name, as specified in `add-countblock`.
- `cb`: Dictionary formatted as Section 3.7.5. Ensure it contains the latest counter by updating `cb` first.
- `subname`: Supplemental text displayed after the counter (e.g., theorem name).
- `count`: Set `false` to disable numbering.
- `lab`: Label for cross-referencing with `@lab`.
- `breakable`: Applies only to this call; set `false` to keep this block on one page.

Example using the `test` counter created in Section 3.7.4:

```
#countblock("test", blocks)[
  1 + 1 = 2
]
```

This is a test 3.1

$1 + 1 = 2$

Alternatively, create a wrapper function:

```
#let test = countblock.with("test", blocks)
#test[
  1 + 1 = 2
]
```

This is a test 3.2

$1 + 1 = 2$

3.7.7 Summary

Use `add-countblock` to extend the registry, pass the entire registry to the template with `countblocks`, and use `countblock` to create individual blocks. Ratchet manages numbering, resets, and references.

Example

Combined example: default blocks use depth `3`, `remark` is detached from the `prop` family with depth `1`, and a new `algorithm` uses depth `2`.

```
#let blocks = set-countblock-depth(cb, "rmk", 1, detach: true)
#let blocks = add-countblock(blocks, "alg", "Algorithm", yellow,
depth: 2)
#let remark = countblock.with("rmk", blocks)
#let algorithm = countblock.with("alg", blocks)

#show: scripst.with(
  // ...
  countblocks: blocks,
  cb-counter-depth: 3,
)
```

Put the registry and wrapper definitions before `#show: scripst.with(...)`.

3.8 Some Other Blocks

3.8.1 Blank Block

Additionally, Scripst provides this type of block without a title, and you can use it by customizing the color.

For example:

```
#blankblock(color: color.red)[
  This is a red block.
]
```

This is a red block.

3.8.2 Proof and ■ (Quod Erat Demonstrandum)

```
#proof[
  This is a proof.
]
```

Proof.

This is a proof.

■

This provides a simple proof environment along with a tombstone symbol.

3.8.3 Solution

```
#solution[
  This is a solution.
]
```

Solution.

This is a solution.

This provides a simple solution environment.

3.8.4 Separator

```
#separator
```

You can use the `#separator` function to insert a separator.

---

4 Conclusion

The above documentation demonstrated Scripst, explained the template parameters, and showed the template effects.

I hope this document helps you better use Typst and Scripst.

You are also welcome to provide suggestions, improvements, and/or contribute code to Scripst.

Thank you for your support of Typst and Scripst!